COMPUTATIONAL EXPERIMENTS WITH STEIN FACTORIZATION IN MACAULAY2

TAKEHIKO YASUDA

Abstract. We implement a prototype of a Stein factorization algorithm in Macaulay2 and test it on several explicit projective maps. The implementation follows the bigraded Hom-module construction in the author's algorithm:

$$C = \mathrm{Hom}_R(R_{\geq r}, R)_{(0,\geq 0)}$$

The module is used to reconstruct a graded coordinate algebra for the Stein intermediate scheme and, where possible, the graph of the connected-fiber morphism. This note is not intended as a formal verification of the algorithm; rather, it records a reproducible computational experiment that demonstrates the practical feasibility of the construction on concrete examples. All calculations are included in the repository and can be reproduced with Macaulay2 1.24.11 or later. The code and this note were written essentially by an AI system, with minor edits by the author; see Use of AI.

1. Purpose and background

For a projective morphism of algebraic varieties

$$f : Y \longrightarrow X,$$

a Stein factorization is a decomposition

$$Y \xrightarrow{h} Z \xrightarrow{g} X$$

in which h is proper with $h_*\mathcal{O}_Y = \mathcal{O}_Z$ (so that h has connected fibers) and g is finite; concretely $Z = \mathrm{Spec}_X\, f_*\mathcal{O}_Y$. The purpose of this project is to turn an algorithm described by Yasuda (2026) into an executable Macaulay2 prototype and to see how it behaves on concrete examples from projective geometry.

The input is the graph of f represented as a bigraded projective scheme. If

$$R = S/I_{\Gamma_f},$$

then the variables in the source block have degree $(1, 0)$ and those in the target block have degree $(0, 1)$. The two gradings make it possible to separate the source direction from the target direction during the Hom-module computation.

2. The computational construction

For a sufficiently large bidegree $r = (r_1, r_2)$, the implementation computes

$$C = \mathrm{Hom}_R(R_{\geq r}, R)_{(0,\geq 0)}.$$

The truncation bound is obtained from shifts in a bounded bigraded free resolution of R over the ambient polynomial ring. This is the computational counterpart of the bound in Corollary 4.3 of Yasuda (2026). The bigraded construction extends the monograded global extension module computation of Smith (2000) to the bigraded setting.

Date: July 2026.

For r large enough, $\mathrm{Hom}_R(R_{\geq r}, R)$ computes the sections of the structure sheaf of $\Gamma_f \cong Y$, so its $(0,n)$-part is

$$H^0(Y, f^*\mathcal{O}_X(n)) = H^0(X, f_*\mathcal{O}_Y \otimes \mathcal{O}_X(n)) = H^0(Z, g^*\mathcal{O}_X(n)).$$

That is, the $(0, \geq 0)$-strand is the section ring of the Stein intermediate Z with respect to $g^*\mathcal{O}_X(1)$. This is why the examples below produce Veronese section rings rather than the usual homogeneous coordinate rings: the polarization on Z is pulled back from X.

The main public functions are:

```
data  = steinHomData(S, graphIdeal, d1, d2, c1, c2)
cData = steinCoordinateAlgebra(data, gammaIndex, baseImages)
gData = directSteinGraph(data, cData)
```

Here `d1`, `d2` are the dimensions of the ambient (weighted) projective spaces of the source and target blocks — that is, block s has variables $x_{s,0}, \ldots, x_{s,d_s}$ — and `c1`, `c2` are the sums $c_s = \sum_{t=0}^{d_s} c_{s,t}$ of the degrees of those variables, as in Proposition 4.2. All examples here are standard bigraded, where c_s is simply the number of variables in the block; for a weighted grading the weight sum must be supplied instead. The arguments are not checked against the degrees of the ambient ring, so a wrong value silently produces a wrong bound. Nothing in the main construction is specialized to the unweighted case — degrees are read from the ring throughout — but no weighted example has been run, so weighted input is untested rather than supported. The first function computes the truncated Hom module and records the bound. The second evaluates selected Hom generators at a nonzero element γ and computes the defining relations of the resulting coordinate algebra. The third computes a graph closure from the evaluated generators in a localization. For examples where the minimal resolution is too expensive, the experimental entry point `steinHomDataAtBound` accepts a supplied bound. Such a computation is explicitly marked as non-certified in the output.

3. Environment and reproducibility

The computations were run with Macaulay2 1.24.11 on macOS (Apple Silicon, 2023). The relevant bundled packages include `Truncations`, `MinimalPrimes`, and `Saturation`. Measured wall-clock times on that machine: `tests/basic.m2` about 4 s, `tests/global-hom.m2` under 1 s, `tests/mori-fiber-space.m2` about 1 s, `tests/blowup-line.m2` about 12 s, so the standard suite takes roughly 15–20 s in total. The separate twisted-cubic benchmark takes about 11 s with the supplied bound.

From the project root, the standard suite is run by:

```
./run-tests.sh
```

The slower twisted-cubic benchmark is run separately:

```
M2 --no-readline --stop -q tests/blowup-twisted-cubic.m2
```

The complete input files are included in the repository. The excerpts below are chosen to show the mathematical input and the checks, without reproducing every elimination polynomial or the full Macaulay2 startup log.

4. Representative Macaulay2 calculations

4.1. **A quadratic map of $\mathbb{P}^1$.** The first test is the finite map $[s : t] \mapsto [s^2 : t^2]$. Its graph is described by $y_0^2 x_1 - y_1^2 x_0$. The following excerpt shows the input and the main output.

```
i = ideal(y0^2*x1-y1^2*x0);
d = steinHomData(S,i,1,1,2,2);
c = steinCoordinateAlgebra(d,0,{x0,x1});

d#"bound"
o = {1, 0}
```

```
d#"steinGeneratorDegrees"
o = {{0, 0}, {0, 1}}

c#"definingIdeal"
o = ideal(p_0*p_1-p_2^2)

isPrime (directSteinGraph(d,c)#"graphIdeal")
o = true
```

The ring of the last ideal is generated automatically, so its variables print as p_0, p_1, p_2; they are the images of x_0, x_1, and of the evaluated degree-$(0,1)$ Hom generator. Writing them as X_0, X_1, z — which is what `tests/basic.m2` does, by rebuilding the map into an explicitly named ring — the relation is $X_0X_1 - z^2$.

Thus the Stein intermediate is the source $\mathbb{P}^1$, embedded as the conic $X_0X_1 - z^2 = 0$: since f is finite, $Z \cong Y = \mathbb{P}^1$, polarized by $f^*\mathcal{O}(1) = \mathcal{O}(2)$, whose section ring is the second Veronese ring of $\mathbb{P}^1$.

4.2. **A Mori fiber space with a finite part.** The next example is the composition

$$\mathbb{P}^1 \times \mathbb{P}^2 \to \mathbb{P}^2 \to \mathbb{P}^2,$$

where the second map is $[x_0 : x_1 : x_2] \mapsto [x_0^2 : x_1^2 : x_2^2]$. The projection has connected $\mathbb{P}^1$-fibers, while the second map is finite. The relevant part of the test input and output is:

```
igraph = kernel graphParametrization;
d = steinHomData(sgraph,igraph,5,2,6,3);
c = steinCoordinateAlgebra(d,0,{y0,y1,y2});

numgens igraph, codim igraph
o = (12, 4)

d#"bound"
o = {2, 0}

dim(c#"ring")
o = 3

hilbertFunction(1,c#"ring"), hilbertFunction(2,c#"ring")
o = (6, 15)

isPrime(c#"definingIdeal")
o = true

fiberComponents = minimalPrimes ifiber;
#fiberComponents
o = 4
```

Each of the four fiber components is a copy of $\mathbb{P}^1$. The computed Stein intermediate is the degree-two Veronese model of $\mathbb{P}^2$, and the connected part is the projection to $\mathbb{P}^2$.

Here $Z = \mathbb{P}^2$ carries the polarization $g^*\mathcal{O}(1) = \mathcal{O}(2)$, so its section ring is the second Veronese ring of $\mathbb{P}^2$, i.e. the subring of $k[x_0, x_1, x_2]$ generated by the degree-2 monomials. Its Hilbert function is $H(1) = 6$ (the 6 monomials of degree 2 in 3 variables) and $H(2) = 15$ (the 15 monomials of degree

4 in 3 variables). The computation reproduces these known invariants, which is what one expects if the algorithm identifies the Stein intermediate correctly.

4.3. **The blow-up of a line in $\mathbb{P}^3$.** We next take the blow-down $\mathrm{Bl}_L(\mathbb{P}^3) \to \mathbb{P}^3$ along a line L, followed by the coordinatewise square map on $\mathbb{P}^3$. The main numerical checks are:

```
numgens igraph, codim igraph
o = (26, 7)

d = steinHomData(sgraph,igraph,7,3,8,4);
d#"bound", #d#"steinGeneratorIndices"
o = ({2, 0}, 8)

dim(c#"ring")
o = 4

hilbertFunction(1,c#"ring"), hilbertFunction(2,c#"ring")
o = (10, 35)

isPrime c#"definingIdeal"
o = true

degree(ssource/ie)
o = 2

#fiberPrimes, all(fiberPrimes,p -> degree(rfiber/p) == 1)
o = (8, true)
```

The Stein ring has the Hilbert values of the second Veronese ring of $\mathbb{P}^3$, as expected for $Z = \mathbb{P}^3$ with $g^*\mathcal{O}(1) = \mathcal{O}(2)$: $H(1) = 10$ (the 10 monomials of degree 2 in 4 variables) and $H(2) = 35$ (the 35 monomials of degree 4 in 4 variables). The computation recovers both values. Note that the presentation produced by the algorithm uses 11 variables, one of the degree-two module generators being redundant as an algebra generator; the quotient ring is nevertheless the expected one.

The exceptional divisor of $\mathrm{Bl}_L(\mathbb{P}^3) \to \mathbb{P}^3$ over a line L is the projective bundle

$$\mathbb{P}(N_{L/\mathbb{P}^3}) = \mathbb{P}(\mathcal{O}(1)^{\oplus 2}) \cong \mathbb{P}^1 \times \mathbb{P}^1.$$

The computation returns a surface of degree 2, in agreement with the Segre quadric; the check is carried out on the source side, independently of the Hom construction.

4.4. **The twisted-cubic blow-up.** The final example is the blow-up of $\mathbb{P}^3$ along the twisted cubic, followed by the coordinatewise square map. It is deliberately retained as a heavier benchmark. Here the source sits in $\mathbb{P}^{11}$, so the automatic Corollary 4.3 bound would require a minimal free resolution out to homological degree $11 + 3 + 1$; that computation did not finish within about ten minutes on the machine used here and was aborted. The test therefore supplies $r = (2, 0)$ and records that this bound is experimental; with the supplied bound the example runs in about 11 seconds.

```
reesI = kernel map(targetRees,trees,{...});
numgens reesI, codim reesI
o = (3, 2)

igraph = saturate(igraphRaw,ideal matrix{flatten rows});
numgens igraph, codim igraph, isPrime igraph
o = (70, 11, true)
```

```
d = steinHomDataAtBound(sgraph,igraph,{2,0});
d#"steinGeneratorDegrees"
o = {{0,0},{0,1},{0,1},{0,1},{0,1},{0,1},{0,1},{0,2}}

hilbertFunction(1,c#"ring"), hilbertFunction(2,c#"ring")
o = (10, 35)

d#"certifiedBound"
o = false
```

The resulting ring again has the Hilbert values of the second Veronese of $\mathbb{P}^3$. One degree-two module generator is algebraically redundant, so automatic graph construction is not yet completed for this example. This is a useful illustration of the difference between a bound that happens to work and a bound certified by Corollary 4.3.

5. Summary of results

In the table below, dim is the Krull dimension of the computed Stein coordinate ring, i.e. $\dim Z+1$ (the ring is the section ring, so it is the affine cone over Z).

Example	Bound	dim	Selected Hilbert data	Graph
$\mathbb{P}^1$ quadratic map	$(1,0)$	2	$H(1)=3$	prime
$\mathbb{P}^1$ cubic map	$(2,0)$	2	twisted cubic	prime
$\mathbb{P}^1\times\mathbb{P}^2$	$(2,0)$	3	$H(1)=6, H(2)=15$	prime
$\mathrm{Bl}_L(\mathbb{P}^3)$	$(2,0)$	4	$H(1)=10, H(2)=35$	prime
Twisted-cubic blow-up	$(2,0)^*$	4	$H(1)=10, H(2)=35$	partial

The star (*) indicates a supplied experimental bound rather than a bound computed from the full minimal resolution.

Interpretation: Across these examples, the implementation reproduces the expected intermediate objects for finite maps, positive-dimensional connected fibers, and a divisorial contraction. The graph ideals produced by the direct localization strategy are prime in the completed examples. The computed Hilbert functions, exceptional divisors, and fiber structures agree with the classical invariants of the corresponding geometry. This is evidence that the implementation behaves as intended on these inputs; it is neither a proof of the algorithm nor a statement about inputs of a different shape.

6. Limitations

This is a research prototype rather than a general-purpose Macaulay2 package. The main limitations are:

- Deep minimal free resolutions can dominate the exact-bound computation
- Finite module generators may be redundant as algebra generators
- Heuristic stabilization gives evidence but does not prove the explicit bound
- The component-selection part of the full algorithm is not automated for every possible input
- Only standard bigraded input has been tested. The paper allows weighted projective spaces, and the main construction reads degrees from the ring rather than assuming weight one, but no weighted example has been run; `certifiedHomogeneousGraph` moreover assumes an unweighted target

- The bound arguments `d1,d2,c1,c2` are trusted, not validated against the ambient ring, although they are determined by it

The calculations should therefore be read as reproducible computational evidence, not as a replacement for the mathematical hypotheses or proofs in the source paper.

7. Use of AI in the implementation

The Macaulay2 code, the test files, and this note were written essentially by an AI system (Claude); the author only made minor edits. The author has read the code and the text and believes them to be correct, but has not verified every line, every intermediate computation, or every claim in detail. Readers should keep this in mind.

Two points nevertheless make large errors unlikely. First, the algorithm being implemented is the one described in Yasuda (2026); nothing conceptually difficult is being attempted here, so the room for the code to silently do something else is limited. Second, the tests do not merely report what the code computes: each example is checked against independently known geometry (Hilbert functions of Veronese rings, the degree of the exceptional divisor, the number and degree of the fiber components, primality of the resulting ideals), and these checks are run automatically by `run-tests.sh`. A serious error in the construction would have to reproduce all of these known invariants by accident.

None of this replaces a proof, and computations marked as non-certified are not presented as proofs.

8. Conclusion

This project shows that the bigraded Hom-module construction can be used in a practical Macaulay2 workflow on several nontrivial examples. The most useful outcome is not a large collection of examples, but a transparent record linking the mathematical construction, the source code, the actual computer input, and the numerical output. The repository is therefore intended as a public technical note and reproducible experiment rather than as a claim of a fully general implementation.

9. References

- **Smith, G. G.** (2000). Computing global extension modules. *Journal of Symbolic Computation*, 29(4), 729–746.
  (Monograded version of global Hom computation, foundational for the bigraded extension.)
- **Stacks Project Authors.** Stein factorization.
  https://stacks.math.columbia.edu/
  (Definition and basic properties of Stein factorization in scheme theory.)
- **Yasuda, T.** (2026). An algorithm for the minimal model program in dimension three. arXiv:2603.13703v2.
  https://arxiv.org/abs/2603.13703v2
  (Sections 4–5 and Algorithm 1; this repository implements Section 5.1.)
- **Grayson, D. R. & Stillman, M. E.** Macaulay2, a software system for research in algebraic geometry.
  https://macaulay2.com/